PROMO BOSS

Never Lose
Another Sale

OUR MISSON

Promo Boss mission is to create an efficient system for employees to increase sales through promo code management.

Our application aims to add values in which employees do not have to go up the chain of command to get promotion codes.

Saves Time

Employees don't have to go up the chain of command to get promotion codes.

Increase Productivity

Manager have access to look at sales data and provide incentive to employee of best performance.

Simple Code Addition

Code addition system is made user friendly. Managers can simply add codes whenever they are avaliable.

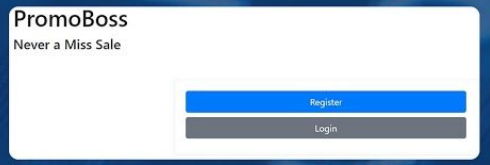
Easy to Retrieve

Code retrieve system is made user friendly. Employee can retrieve claimed codes whenever possible.

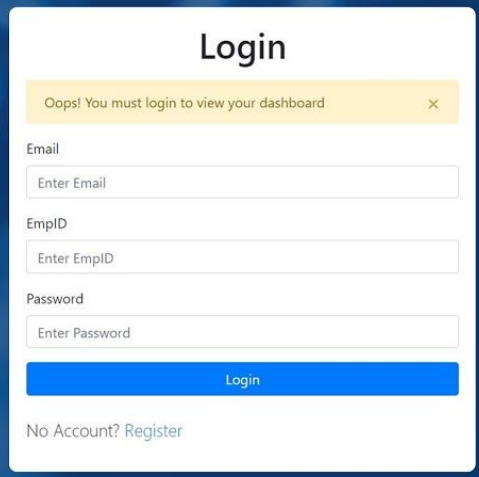
PRIVACY MATTERS

Even our admins don't know your password. So reset assure you credentials are safely stored and authenticated with passportJS, the same authentication framework used to authenticate emails, Twitter, Facebook, etc.



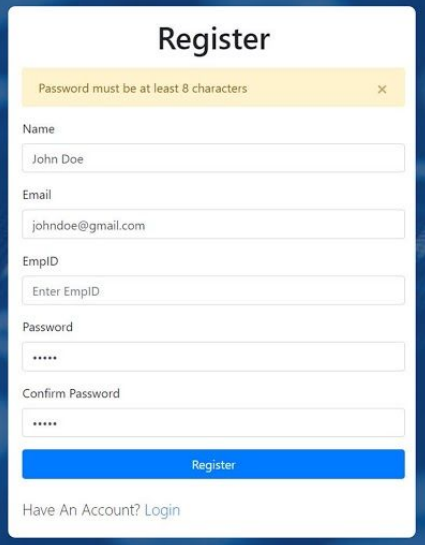


Your data is safe with us.
Only you can access your account.



PRIVACY MATTERS

We can customize the software to your needs. Specific password requirements, primary key based on email or employee ID, your choice.



Light-weight & portable! Your data is securely held on thecloud and your web app is accessible from everywhere

MongoDB Atlas

Move faster with a cloud MongoDB service. Built for agile teams who'd rather spend time building apps than managing databases. Available on AWS, Azure, and GCP.

CONTACT US

hbhargav@ryerson.ca

mwashraf2@gmail.com

abo.said@ryerson.ca

ruhao.lou@ryerson.ca

MORE INFORMATION

www.facebook.com/promocode/

PromoBoss

Never Lose Another Sale

Hemanshu Bhargav 500588413
Ruhao Lou 500692743
Maria Ashraf 500407958
Said Abo Muslem Said Muhammad 500652442

Table of Contents

Chapter A: SDLC Activities

- Proposal 1
- Background 2
- Business Case 3
- Time Schedule (HR, Labour Cost & Meeting Minutes) 4
  - Gantt Chart 4.1
  - Milestones 4.2
  - SCRUM Sprint Backlog Template 4.3.1
- Sprint Burndown 4.3.2
- Requirements analysis 5
- Use Case diagram 5.1.1
- Sub-function: Login system Use Case Diagram 5.1.2
- Subfunction: Code Usage Data Use Case Diagram 5.1.3
- Swimlane Activity Diagram 5.1.4
- Design 6
- Sequence Diagram 6.1
- UML Class Diagram 6.2
- B-Interface Design 6.3
- UML Communication Diagram: Claim Code Scenario 6.4
- C-Component Level Design 6.5
- Architectural Design Style/Taxonomy 6.6
- Pipe and Filter/Data Flow Architecture 6.6.1
- Model View Controller 6.6.2
- Testing 7
  - Test Specification 7.1
  - Technical Reviews 7.1.1
  - Testing Strategies 7.1.2
  - Testing Sequence 7.1.3
  - Verification Testing 7.1.4
  - Verification Test Cases 7.1.5
  - Validation Testing 7.2.1a
  - Validation Test Cases 7.2.1b
  - System Testing 7.3.1
  - Regression Testing 7.3.2
  - Unit Testing 7.3.3
  - Test Results 7.4
  - Verification Tests Passed 7.4.1
  - Documented Errors 7.4.2
  - Error/Obstacle #1: System/Compiler Errors (Solved & Overcome) 7.4.2a
  - Error/Obstacle #2: Unforeseen Circumstances (Solved & Overcome) 7.4.2b
  - Error/Obstacle #3: Integration 7.4.2c

- Error/Obstacle #3: Button Mapping 7.4.2d
- Metrics 7.5
- Size Oriented 7.5.1
- Final Product 8
- Abilities 8.1
- Constraints 8.2
- Missing Features 8.3

Chapter B: Umbrella/Support Activities

- HR Section 9
- Hours Worked Per Team Member 9.1
- 3-Week Plan 9.2
- Earned Value Analysis 9.3
- Risk Mitigation Monitoring Management 10
- Quality Assurance 11
- Lessons Learned 11.1
- SW Engineering models and principles 12
- Methodology: Agile 12.1
- Process Framework: Scrum 12.2
- Methodology Revision: Why SCRUM Proved to be Nonoptimal 12.3
- Daily Scrum Questions 12.4
- Integration 13
- Reflective Assessment 14
- Team Reflection (Week 5) 14.1
- Individual Reflection 14.2
- Team Reflection (Week 11) 14.3
- Going Forward 14.4
- Challenges 14.5
- Appendix 15
- Meeting Minutes 15.1
- Hours Worked 15.2
- Tech Specs 15.3
- Bibliography 16

Chapter A: SDLC

1 Proposal

The idea is to create an efficient system for employees to increase sales through promo code management.

A universal web application which can facilitate the managing of organization specific codes used in marketing, customer service and operations. The application aims to be an organizational tool for a critical, time-consuming element in many jobs, particularly for service roles. The application will have login functionality for multiple organizations, database storage of codes, real-time data refresh and sorting of codes which have been used, expired or to be added.

A. Boundary of the Topic

a. Business boundaries: Telecommunications companies only. Private commercial use which is limited to sales employees and their managers
b. Technical boundaries: web based, alphanumeric codes only
c. Not a productivity tool in and of itself, but one that enables better organization and data not presently available in these companies

B. Big Picture

a. Login system module
b. Database module
c. Business logic module
d. Learning module (for new tools)
e. UI module

C. Inputs and Outputs of Proposed Service

a. Input: Codes to populate database. Employees will be able to create requests for codes to the manager.
b. Output: List of codes with used codes, expiry. etc. ranked at top. Code generator for manager, email notification when system runs out of codes.

D. Functions it Will Accomplish

a. Efficient tracking of codes
b. Tracking of sales by employees
c. Allow employees to request codes
d. Manage the Promo Code process

E. Resources and Constraints

a. Project Deadline: 06/12/2019
b. Proposed completion Date: 29/11/2019

c. Allotted Weeks: 11
d. Workload distribution (Overlapping Roles): 1 leader, 2 developers, 1 designer, 1 documenter/writer

2 Background

The idea arose from a problem that one of the group members regularly faces at work. Often during sales and customer service operations at internet/telecommunications service providers, single use promo codes require various permissions from the corporate chain and the ground-level agents lose sales. The proposal is to create a centralized system to increase efficiency, time management and boost sales through the streamlined automatic management of promo codes.

3 Business Case

The idea is to create an efficient system for employees to increase sales through promo code management. A universal web application which can facilitate the managing organization of specific codes used in marketing, customer service and operations. The application aims to be an organizational tool for a critical, time-consuming element in many jobs, particularly for service roles. The application will have login functionality for multiple organizations, database storage of codes, real-time data refresh and sorting of codes which have been used, expired or to be added. The value it adds is that employees don't have to go up the chain of command to get promotion codes. It is a productivity application for employees. The value it adds is saved time. Employees do not have to rely on their supervisor's availability to send them codes the moment they need it. Supervisors can simply add codes to the system when convenient for them, and employees can retrieve when needed as well. The final consumer of this product are sales employees of telecommunication companies, and their managers/bosses. The sales employees would have access to their profile, and list of codes. The managers would have access to add codes as well as look at sales data by employee ID. The target sector for this product is private telecommunication companies that give out promotional codes to their sales employees to incentivize customers to buy.

4 Schedule/Time

4.1 Gantt Chart

A Gantt Chart is a type of project schedule illustrated in bar chart. It is one of the most well known and popular ways of showing tasks or events displayed along with time. Gantt Chart helps break down project in smaller parts that is manageable for programers to complete project on time.

Following page (figure 1) is our Gantt Chart, broken down into six parts (Brainstorm and Five Milestone).

4.2 Milestones

Brainstorm(Sep 20th - Oct 4th)

This stage of Gantt Chart is planning period. Business case, such as target audience and market demand are mapped out in order to have a clear goal for the project ahead. This is where market research takes place. Team members will meet up and discuss customer demand and need to come up with program statement.

Milestone 1: Dashboard (Oct 11th - Oct18th)

Here the the static pages were developed. Research was performed and it was discovered that the JQuery tool, Datatables, could be adapted to fit our needs. Datatables come built-in with standard methods for sorting by column (based on given parameters such as alphabetical or type, i.e. Boolean), a fully-functional search bar and flexible data input options. The flexible data input option was particularly of interest due to our necessary integration with a database. The ability to input JSON data, which MongoDB Atlas exports, since MongoDB is a collection of JSON documents, theoretically allowed for seamless integration between the front-end table, and the back-end database.

Milestone 2: Login System (Oct 11th - Nov 1st)

This is where we develop our main pages such as: Login Page, Register Page, Employee Page, and Manager Page. The challenges at this stage is technical base. We will be implementing input/output functionality to each page before connecting to database that store user's data. This stage of the timetable, we are moving on to Database implementation. The goal is to make a Database that holds all information input by the users (Employee or Manager), saved and organized until when it is being called to use again by the users. In our project's case, This will include functionality such as tracking code system and requesting code. This is so that when Employee wants to claim a promo code, it will save that information as "in use" in the database. Same case for Manager. When there are not enough code in the database, the manager can update new code into the database.

Milestone 3: Learning (Oct 11th - Nov 1st)

Learning is an ongoing process during the technical development. Both web page and database technical applications need to be learned as we develop the program.

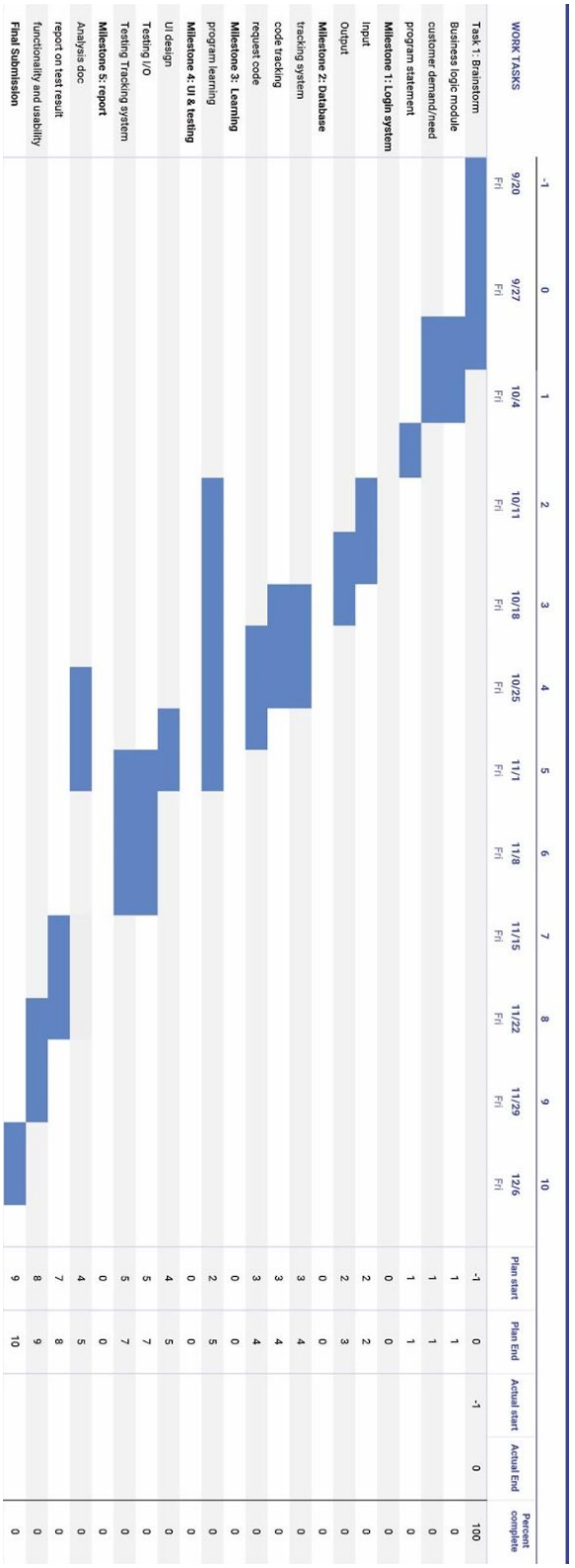
Milestone 4: Testing (Oct 25th - Nov 15th)

Once user interface (how it appears on the website) is done, then comes the most important processes in the project development, testing. In order to catch all errors before releasing, it is mandatory to test the program multiple times, ensuring all errors are addressed before releasing. In our Promo code case, this includes testing user input/output and data tracking system.

Milestone 5: Report (Oct 18th - Dec 6th)

Report will take place between the testing phase and submission. Report records the process of making of product that can be further reviewed in the future.

Figure 1: Gantt Chart



4.3.1 SCRUM Sprint Backlog Template

Agile Sprint Backlog is a technique used to organize the project by breaking work into several parts. These tasks conducted within a sprint is adaptable to problems and countered with solutions planned within the time by team members. Figure 2 is our Project's Sprint Backlog Template. It is clear when comparing estimated time to actual time, we are far behind schedule (17 hours in total to be exact). If we look closely each task are behind schedule by an average estimate of 45 minutes per task assigned. This tells us that in every part of sprint break backlog, we underestimated problems we encounter during the process, thus led to current underachievement.

4.3.2 Sprint Burndown

Stated in the Agile Sprint Backlog Template, we underestimated our workload in the limited time resulting 17 hours delay in the work schedule. This graph shows the estimated workload and time versus the actual workload and time. There is a significant difference since falling behind in the first week. The workload piled up to the following weeks and thus resulted in our current situation.

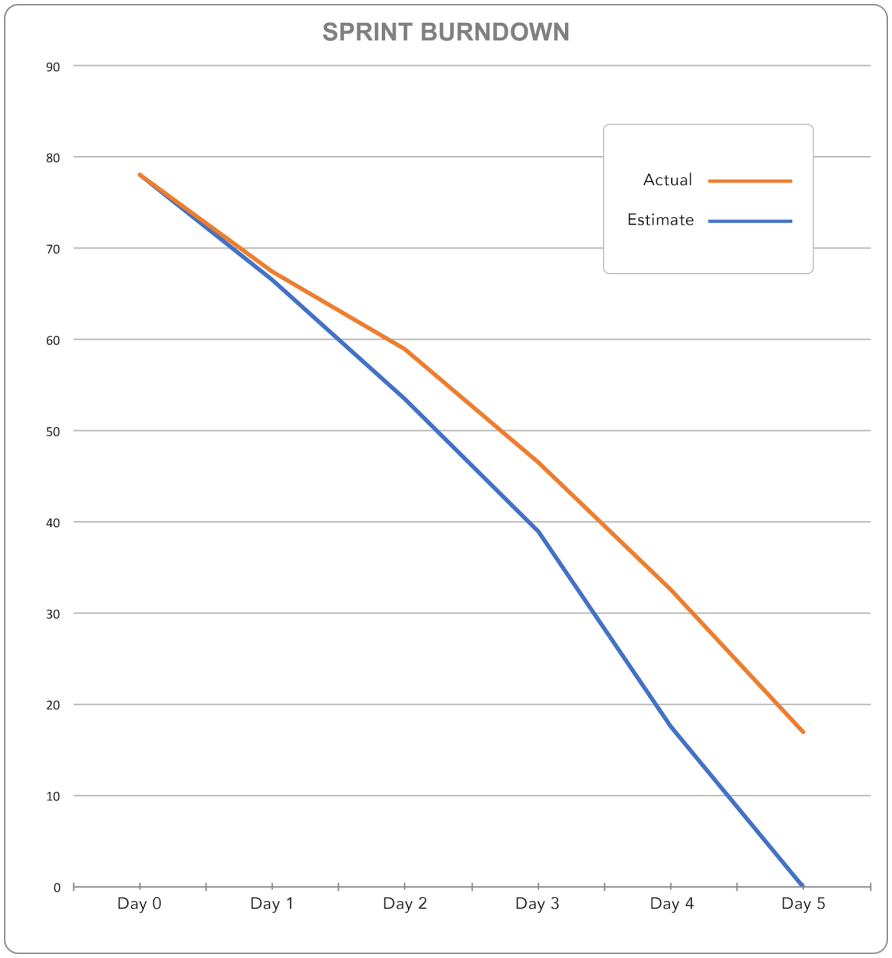
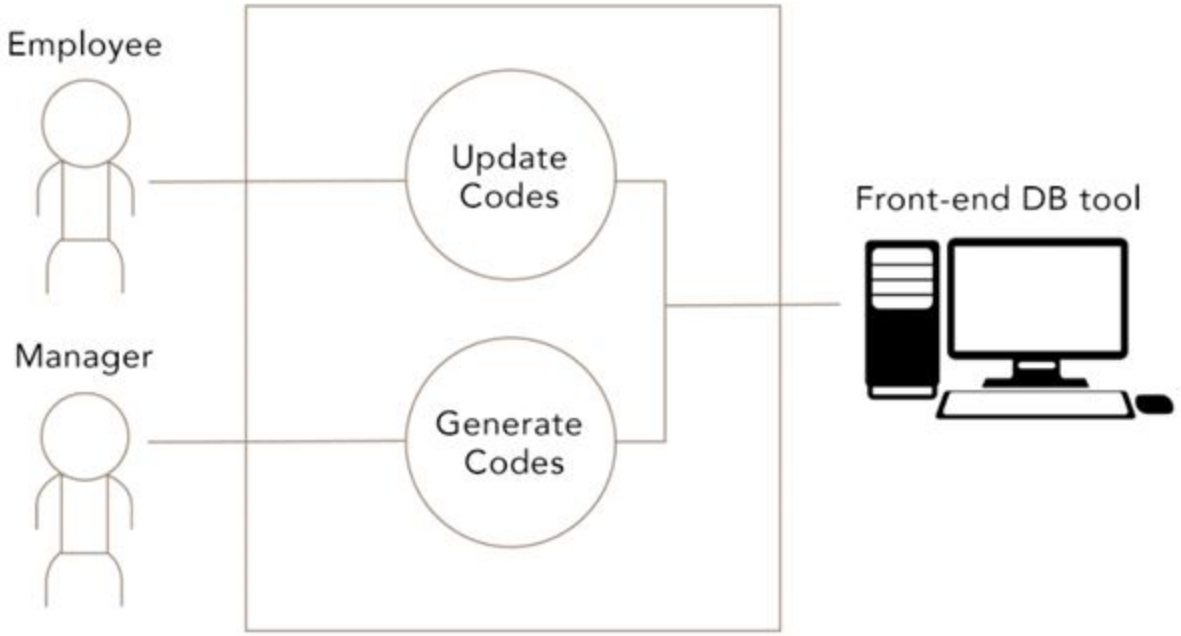


Figure 2: Agile Sprint Backlog Template

BACKLOG TASK & ID	STORY POINTS	ASSIGNED TO	STATUS	ORIGINAL ESTIMATE	DAY 1	DAY 2	DAY 3	DAY 4	DAY 5	SPRINT REVIEW
User Story #1	8			7	3	1.5	1.5	0	0	6
Task		Brainstorm		3	1	1	0	0	0	2
Task		Business Logic Module		3	1	1	0	0	0	2
Task		Customer Demand/Need		1	0.5	0	0	0	0	0.5
Task		Program Statement		0.5	0	0.5	0	0	0	0.5
User Story #2	1									
Task		Login System		3	1	0.5	0.5	0	0	2
Task		Input		3	1	0	2	0	0	3
Task		Output		2	1	0	0	0	0	1
Task		Database		5	2.5	1.5	0.5	0	0	4.5
User Story #3	5									
Task		Learning		8	0.5	1.5	1.5	2	1.5	7
Task		Tracking System		3	0	0.5	1.5	1	0	3
Task		Code Tracking		1.5	0	0.5	0.5	0.5	0	1.5
Task		Request Code		2	0	0	1	0.5	0	1.5
User Story #4	8									
Task		Report		9	1	2	0.5	2	1.5	7
Task		Testing Input		6	0	0	1	1.5	1.5	4
Task		Testing Output		6	0	0	0	2	2	4
Task		UI Design		0.5	0	0	0	0	0.5	0.5
User Story #5	3									
Task		Analysis Doc		2	0	0	0	1	1	2
Task		Functionality and Usability		6	0	0	1.5	1	2	4.5
Task		Report on Test Result		9	0	1	0.5	2	2.5	6
Task		Final Submission		0.5	0	0	0	0.5	0	0.5
TOTAL				78	11.5	10.5	12.5	14	12.5	61
ACTUAL REMAINING HOURS				78	66.5	56	43.5	29.5	17	
ESTIMATED REMAINING HOURS				78	66.5	53.5	39	17.5	0	

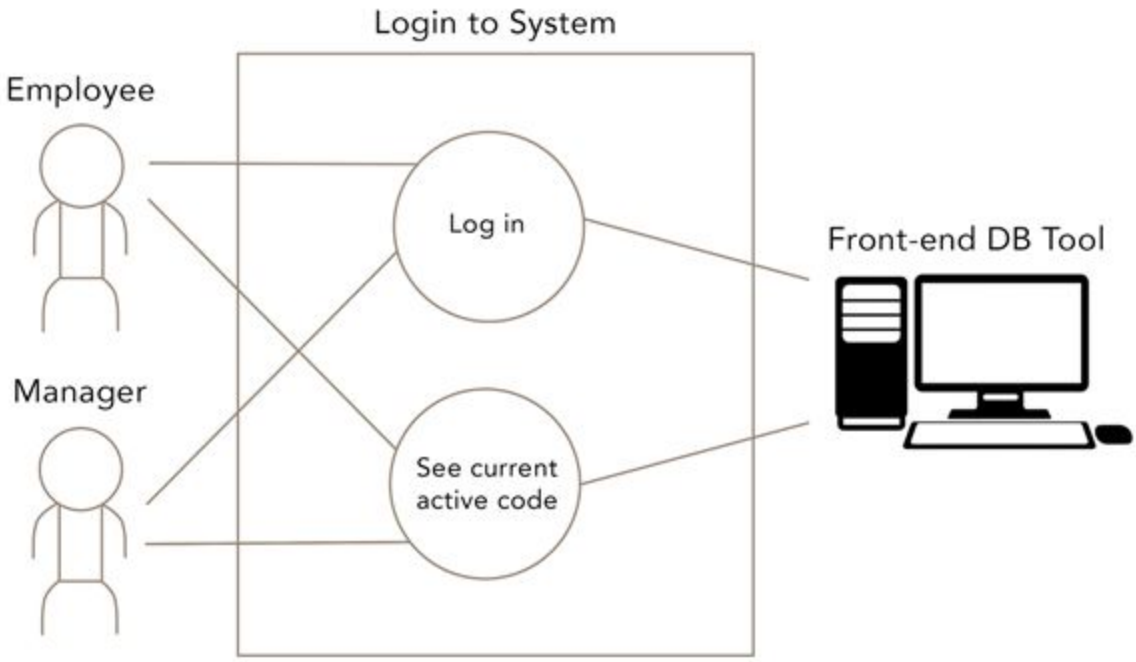
5 Requirement Analysis

5.1.1 Use Case diagram



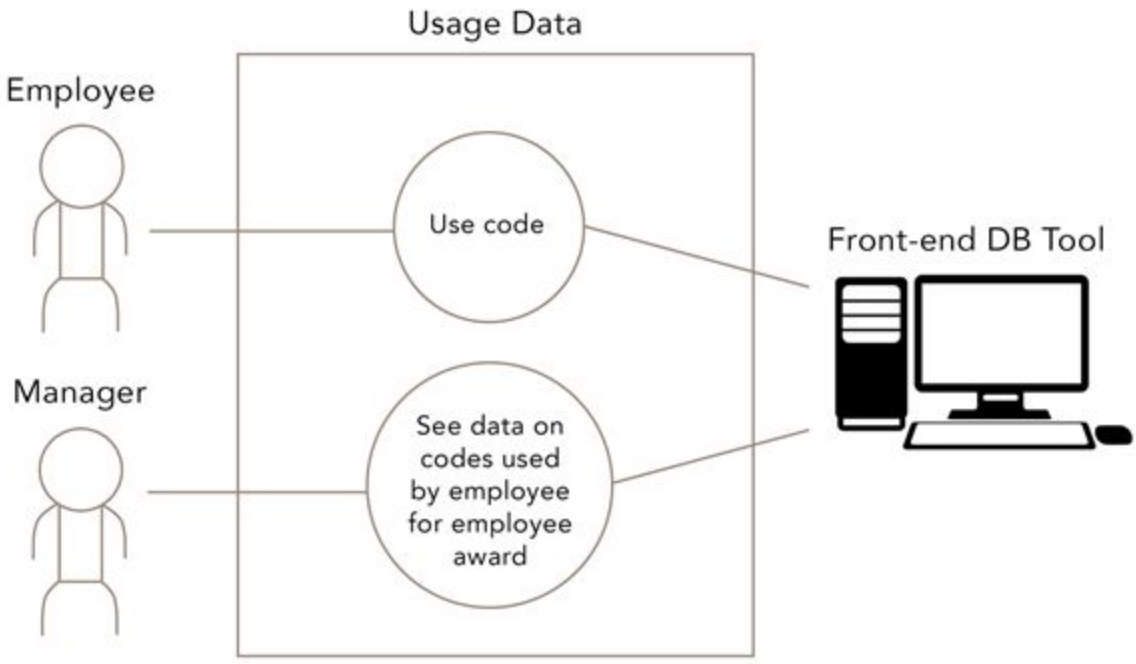
This is a diagram showing the interactions between the primary users of our system and our code database in the most standardized form. In particular, it shows how when an employee user has signed in, they are able to “Update” the codes that are stored in our database system using the Front-end Database Tool, which is essentially the main employee screen. By updating, we mean that when the employee claims an available code from our database, the status of that code will be updated to “Used”, meaning it will not be available for redemption again. As for our second user type, the managers, when they sign login to our system they can “Generate” new codes for our database based on business needs. Once again, the manager is also presented with their own version of our Front-end Database Tool which presents the necessary options exclusively to a manager.

5.1.2 Sub-function: Login system Use Case Diagram



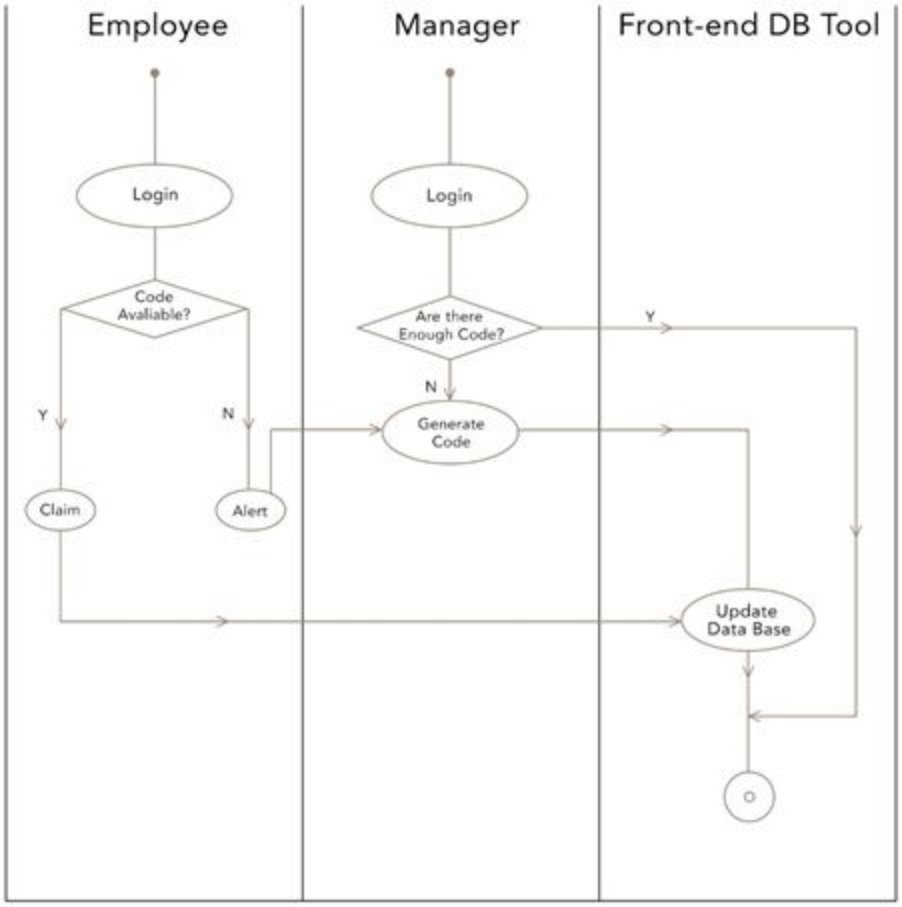
In this diagram we can see the interactions between our different users on our “Login” system. When our users access our web application, they will first be presented with our logging in page. When a user has entered their credentials, they will be treated to their own version of our Front-end DB Tool which displays our active codes located in our database.. When a user of “Employee” account type has logged in, they will be presented with a list of our active codes, from our code database, which are marked as “Available”. When a user of “Manager” account type has logged in, they will be shown with an alternative view of our active codes present in our codes database which focuses on letting them know if there are enough codes marked “Available” left in our system.

5.1.3 Subfunction: Code Usage Data Use Case Diagram



This is a diagram of our second subsystem which shows the interaction between our different users and our Front-end Database Tool. When a user of “Employee” account type has logged in, they are able to “Use” the codes available from our code database which is presented in their version of our Front-end Database Tool. By “use” we mean that the employee can claim a code from the list of the codes marked as “Available” on our code database and use it for their transaction. When a user of “Manager” account type has logged into our system, they are presented with statistics on the codes stored in our codes database. For business case needs, the manager user is able to see which employees have claimed the most amount of codes, which in turn shows which employees have made the most sales, and so then the manager user can proceed to give promotions or other incentives to the said employees as needed.

5.1.4 Swimlane Activity Diagram

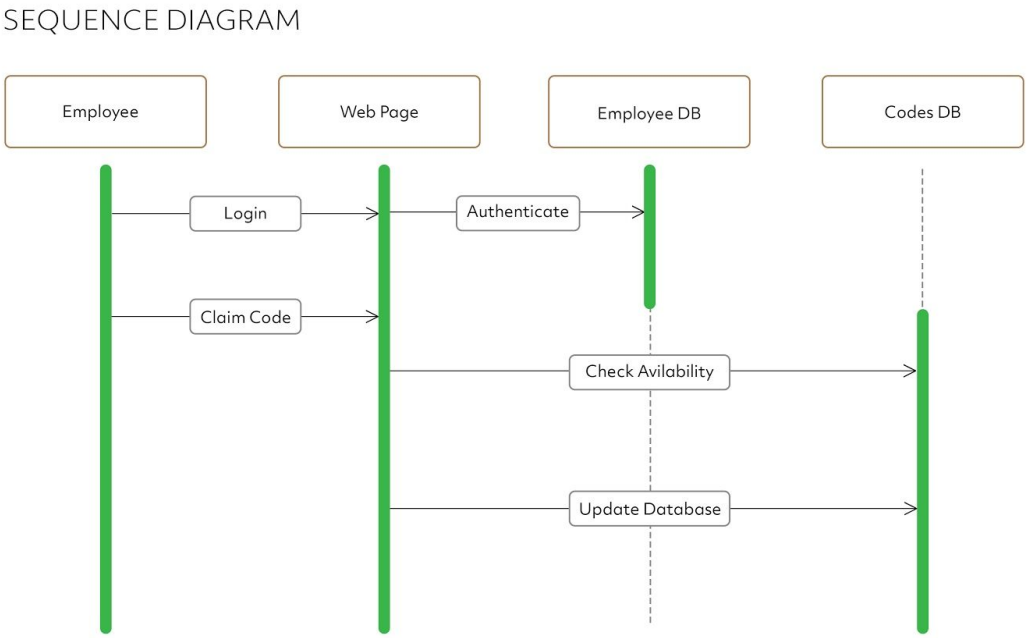


This is a Swimlane Diagram for our Promo Boss web software. The circular bubbles represent actions while the diamonds represent questions and they are connected to each other by lanes which essentially show the flow of interactions of our software users. The lanes following the diamonds are labelled either Y or N which represent “Yes” or “No” answers respectively and direct the lane to the next stage. The sequence for our data flow starts with a user logging in and then being greeted by either the employee or manager screen respectively. The employee users will see if there are any codes available for them to use; if yes then they can press claim on the code which in turn will update our code database by setting switching the status of the claimed code from “Available” to “Used”. If they do not see any available codes, they can “Alert” the manager who can see chose to generate new codes. When a user of “Manager” account type has logged in, they can see if there are enough codes available and can enter new ones if requested or needed. Once new codes are generated, our codes database will be updated. The Front-end Database Tool column represents the user facing webpage

representation of our embedded codes database. Once either user type is done their tasks, their lanes lead to the small circular bubble which represents that their task is finished and they may exist.

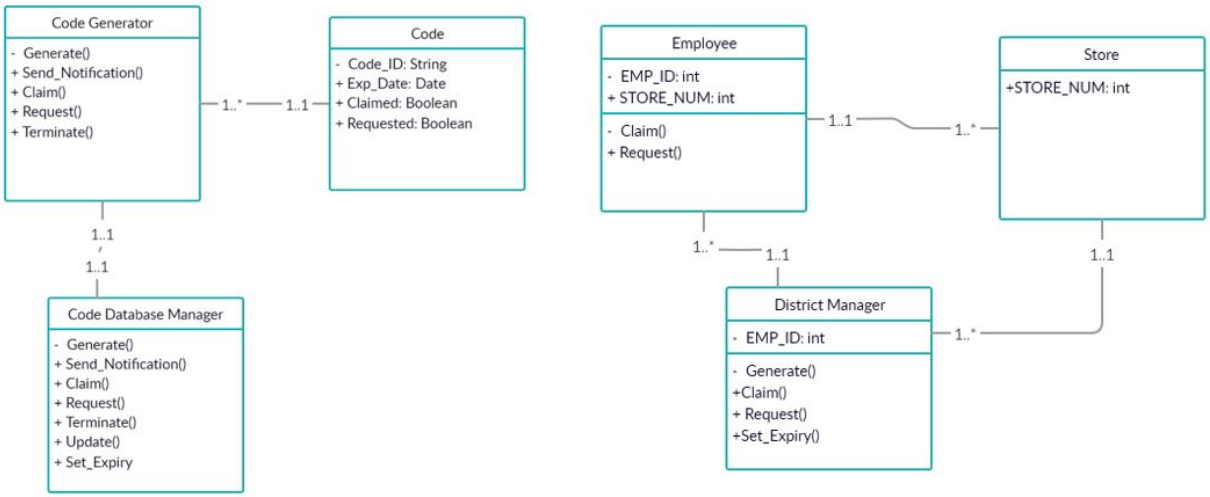
6 Design

6.1 Sequence Diagram



Above we see the Sequence Diagram for an “Employee” user type interacting with our web application. The subheadings are listed in sequence from left to right based on the order they will initiate. Our web page is where all the interactions take place and so we can see that all the actions an employee attempts will occur on the web page designed for them to see while the actual action is sent to the next appropriate stage. When a user decides to “Login”, our Employee Database will be responsible for checking the inputted login query and ensure that the information is valid before allowing the user access to go further. When an employee does sign in, they are able to “Claim” a code, which will only be possible by checking if there is any available codes in our Code Database.

6.2 UML Class Diagram



This is a type of Unified Modelling Language diagrams called a Class diagram. Class diagrams are used primarily to document during software development projects hence why it is a good fit for our PromoBoss web application. The goal is to get 6 classes running which consist of two groups, one group that deals with just the code database and the second which consists of different user types. Code Database Manager class has the methods to modify the codes stored in our database such as generating, claiming, deleting, as well as the means to handle code requests. If needed, the methods in Code Generator class are called to generate new codes codes based on the criteria set under the Code class. The Employee, District Manager, and Store classes are essentially the different types and attributes our users have on our application; employees and managers each have unique IDs which define what functions they can perform on our site, such as being able to generate a code or not.

Employee

- Can claim codes and send request to Manager if there aren't enough codes available

Manager

- Can generate codes, set expiry dates for them and accept requests

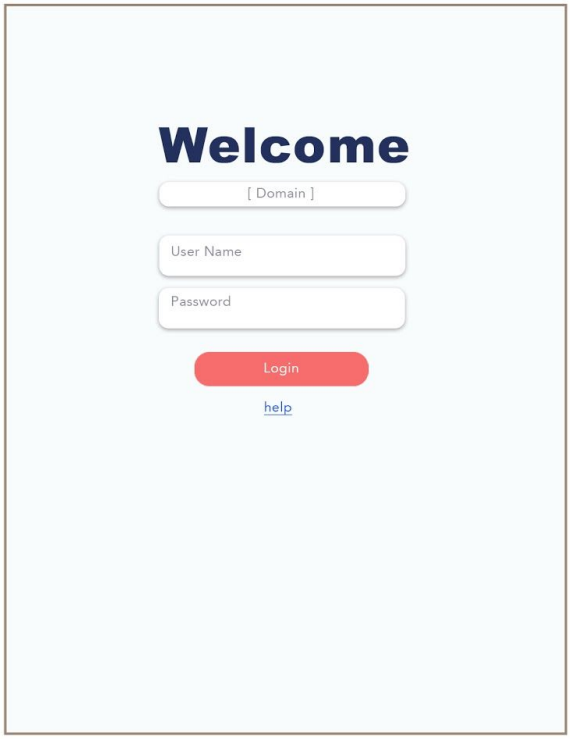
Code Generator

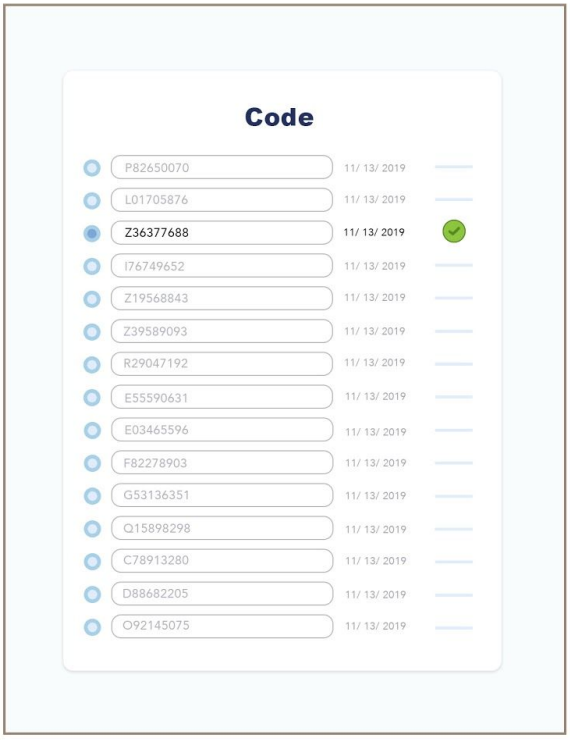
- Can accept request to generate codes and send codes to database

Code Database Manager

- Update list of codes available to remove used codes and accept new codes from Generator

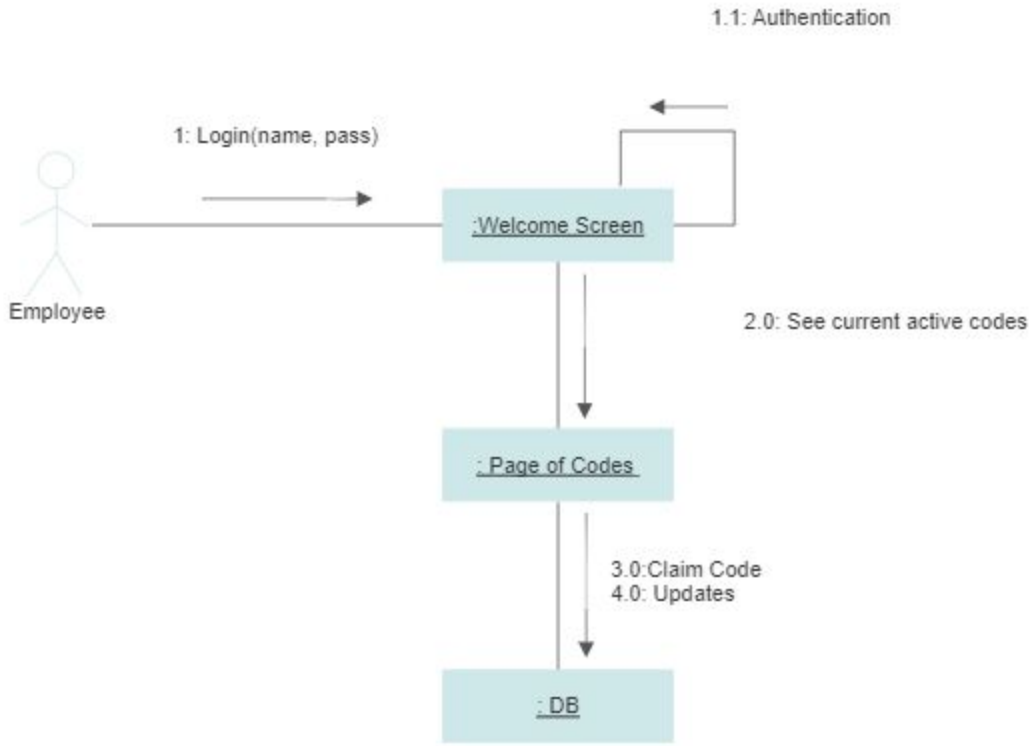
6.3 B-Interface Design





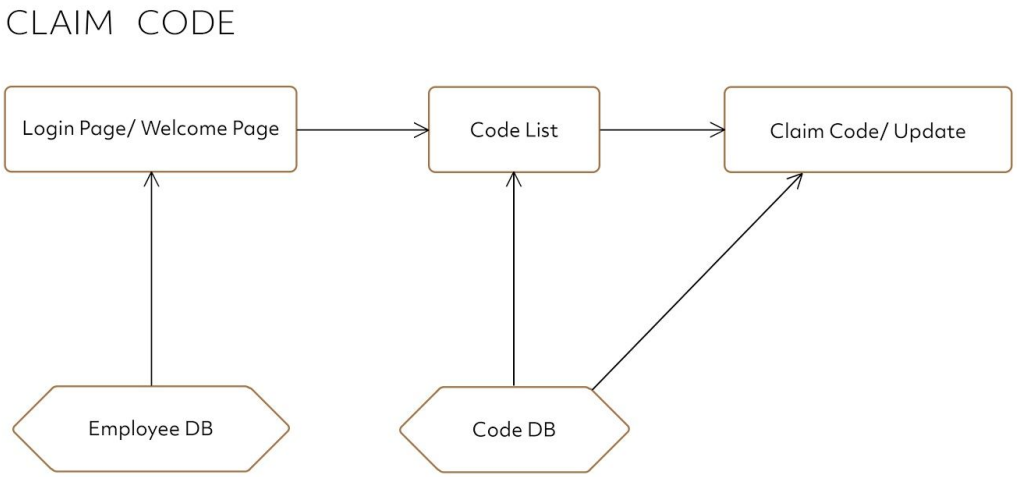
This is a concept Landing Page with a login system and also a sample Page of Codes with expiry dates that a user of “Employee” account type can see. The goal is to get the finale pages formatted using CSS to look this simple and user friendly.

6.4 UML Communication Diagram: Claim Code Scenario



In Claim Code Communications Scenario, an employee logs in at the welcome screen using their employee ID and password. It is authenticated and the actor does not see any changes until the authentication is successful. Employee then sees list of codes which are active (not expired). The employee can claim codes and this will be reflected in the Code DB.

6.5 C-Component Level Design



In Claim Code Component Level design, you can see that the login page is connected to the employee DB in the back-end. Login page leads to the codes list page where the code can be claimed. Both these components are connected the Code DB in back-end.



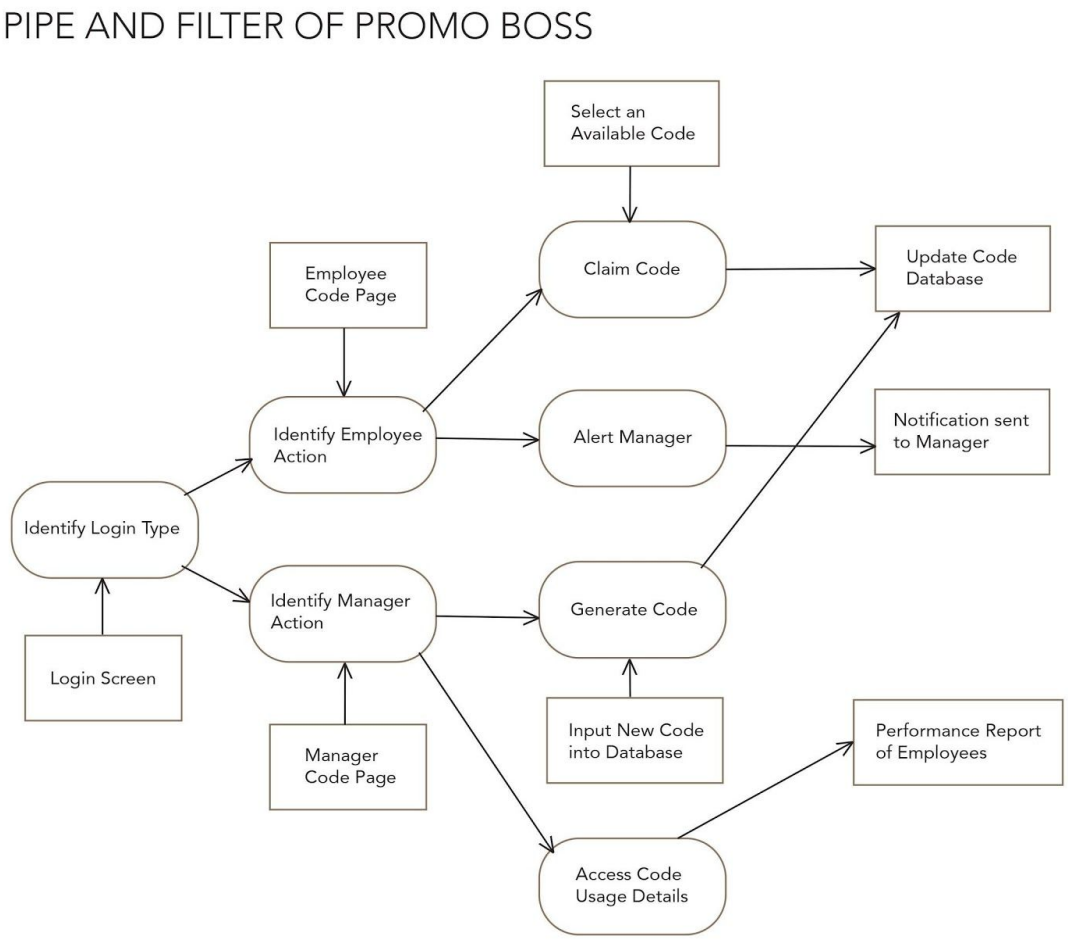
In this chosen Analytics sub system, the login page is connected to the employee DB in the backend. However this scenario is regarding the Manager actor. Manager is able to login, and chose to see performance indicators to pass out awards or incentives. Manager can choose to reward individual

employees across the organization who are top performing, or stores that are top performing. These performance indicators are connected to Analytics DB in the backend.

6.6 Architectural Design Style/Taxonomy

6.6.1 Pipe and Filter/Data Flow Architecture

The processing of the data in a system is organised so that each processing component (referred to as filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.



This is a Pipe and Filter Data Flow Architecture Diagram for our Promo Boss web software. The circular bubbles are the filters in which actions can take place and whose outputs are piped to the filters that follow. Accompanying the filters are brief descriptions of the location or instance in which the said filters take place. The sequence for our data flow starts with a user logging in and then being greeted by either the employee or manager screen respectively. The employee then can either claim an available code or have the ability to alert the manager for a new one. The manager can see if there are enough codes available and can enter new ones if needed, they may also check the performance of their employees by

accessing the statistics of their code usage. Any changes to the codes list causes an update to the code database.

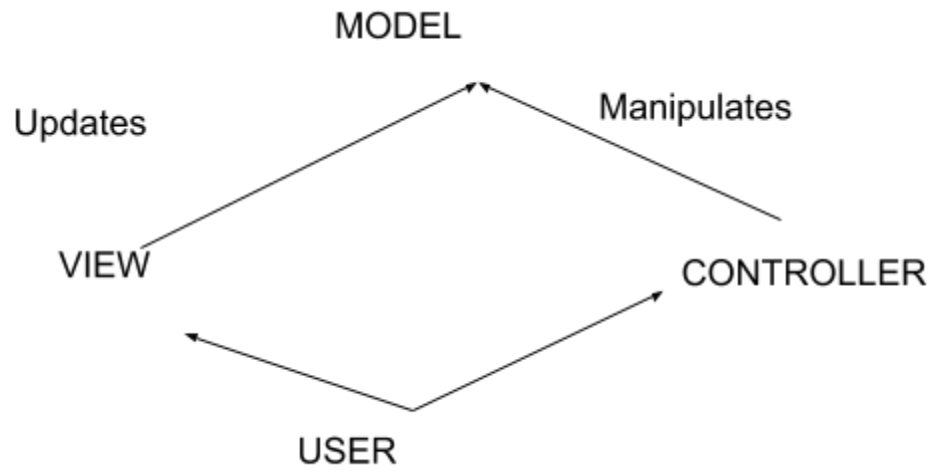
Repository Architecture

The repository pattern is one where all data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.

A repository architecture was a self-evident choice for the design and analysis of PromoBoss, due to both the conceptual layout of the database, as well as the

6.6.2 Model View Controller

The “Model-View-Controller” (MVC) pattern: Separates presentation and interaction from the system data. the system is structured into three logical components that interact with each other. The model component manages the system data and associated operations on that data. The “View” Component defines and manages how the data is presented to the user. The “Controller” component manages user interaction such as key presses and mouse clicks and passes these interactions to the view and the model.



7. Testing

7.1 Test Specification

7.1.1 Technical Reviews

To begin the testing process, we conducted a technical review amongst ourselves. Due to a lack of customers to engage with, we had Abo, our case consultant, play the role of the customer... All members were present for the meeting, which took place during one of our Sprint review meetings.

7.1.2 Testing Strategies

Dominant Testing Strategy: Regression Testing

Due to the nature of webapps, data flow is continuously evolving and the slightest change does not treat users with warnings. As a result of working through text editors (rather than full-fledged IDE's) it is even more imperative that the app be tested after every reasonable logical change in the source code occurs. Therefore, without knowing it consciously, our team employed regression testing from the very beginning of development. Regression testing involves testing all updates after each change. To thoroughly vet our program, we tried different browsers and operating systems to ensure that our code is robust and reliable across multiple recognized platforms. Both major operating systems (Windows, MacOS) were utilized for development to ensure portability (although Windows is the most likely candidate for deployment due to customer preference). A plethora of browsers are also being tested, including Chrome, Firefox and Microsoft Edge. Each and every change was tested by both developers, and each major changes were branched as versions (inspired from version control system concepts).

Unit testing/Web-App Testing

Unit testing is a method of low-level testing which tests all atomic operations and inputs...Fortunately, our web app requires very little user input, aside from the hypothetical input of data and users' logging in with their credentials. Our login system used industry standard passport js for authentication against our database (MondoDB Atlas). After trying multiple user registrations, with varying inputs, we were not able to break find any erroneous input. However, although we successfully, thoroughly vetted our error handling system, we would like to mention that one of our inputs was simply a stub, the employee ID, which was an additional parameter which would have liked to authenticate users against, if the time were available.

7.1.3 Testing Sequence

The testing sequence that we followed was as follows:

1. Regression testing was begun and repeated until the very end of development. As our app had database updated with each execution of the app, it was essential that each and every server refresh was tested. This was done using a developer tool pre-existing in our core framework, nodeJS. This tool or “dev dependency”, forces server refreshes with each change involving the Javascript files.
2. Verification/Unit Testing: Once the application was developed, verification and unit testing began simultaneously.
3. Validation Testing: Once our system passed our robustness tests (which, fortunately, it did), we tested to ensure our proposed user requirements were complete.
4. System Testing: Finally, we performed tests that were not readily apparent. At this stage of testing, we began

   (Usually begins at the Component level and works “upward [structured design]/ outward [OO design],”toward the integration of the entire system.) ← ASK PROFESSOR ABOUT PLACEMENT

7.1.4 Verification Testing

Verification testing is a standard testing strategy employed industry-wide for its accepted efficacy. Verification testing involves .... For our project specifically, verification testing was employed to ...

7.1.5 Verification Test Cases

Requirement: Only store Manager and Employee can login to database.

Test:

- Random inputs to a login page for both employee and manager username and password.
- Try if a user who is not staff member can create login info.
- Both test should deny access with Error.

Requirement: If the data will successfully alert when code is all out.

Test: Feature INCOMPLETE

- set a random employee user. have all codes set to “Taken”. Check if program sees all codes and get result of default.
- Check if Generator receive message of default

- Set a random manager user. check if it recognize that there isn't enough code when login. Compare if the message sent to generator from employee is the same result as when manager login to see Generator code.
- remove manager and employee account.

Requirements: if there are enough codes available, will it send a message to the database?

Test: Cancelled (Feature Omitted)

- set a random employee account. lick a random available code and press claim. Check if claim message is sent to database/if code in database update as used?
- set a random manager account. click yes button from the code generator and see if that message is sent to the database.
- remove manager and employee account.

7.2.1a Validation Testing

Validation testing is testing strategies during the development process. Unlike Verification testing, Validation testing is about building software that is traceable to customer requirements.

7.2.1b Validation Test Cases

Requirement: Both employee and manager must be able to register in the system. Test ● Make a random employee account using the register page. ● check if information registered by employee are saved in the database. ● Make a random manager account using the register page. ● check if information registered by manager are saved in the database ● remove manager and employee account.
Requirements: Web page need to be immediately logged in after hitting login button. Test ● Set a random employee account. ● Login with new employee account on old computer to see how long it will take to bring employee to their page. ● Set a random manager account. ● Login with new manager account on old computer to see how long it will take to bring manager to their page. ● Remove manager and employee account.
Requirements: Webpage need to land on the rightful places for rightful users. Test

● Set a random employee account. ● Login with new employee account and check if it lands on the rightful platform. ● Set a random manager account. ● Login with new manager account and check if it lands on the rightful platform. ● Remove manager and employee account.
Requirement: Webpage must be easily understood by the user (must be user friendly) Test ● Set a random employee account. ● let employee login to the system and see if he or she can understand how system works without any printed instructions. ● Set a random manager account. ● let manager login to the system and see if he or she can understand how system works without any printed instructions ● Remove manager and employee account.

7.3.1 System Testing

System testing is a testing of fully integrated and complete software product. System testing is a series of different tests within the product and to check exercise of complete computer base system.

(rephrase) System testing does the fully integrated applications including external peripherals in order to check how components interact with one another and with the system as a whole. This is also called end to end scenario system.

Black Box testing

Black box testing is testing performed by individuals external to the development team. During black-box testing, all implementation details are hidden from the testing team (or in our case, one of the non-developers) and the user interacts with the system to ensure it works from a user's point of view. To ensure an authentic black-box test, the implementation details were not discussed with our black-box tester.

Test Cases using black box testing

- Make test system for employee that will register, login, and claim a code from their logged in platform.
- Make test system for manager that will register, login, and check number of codes available in the database
- Make test system for manager that will register, login, and update manual code into the database.
- if either each part of the action process fail, the test system will alert problem in that specific field that failed the test cases.

7.3.2 Regression Testing

Regression testing is defined as a type of software testing that confirm if new features are running properly. we will be using prioritization testing with the following three reasons.

1. We have fewer field for our product but usage of these few platforms plays a significant role in our product
2. less costly if products are paid
3. saves time to test on more important functionality of the product than the less important ones.

Test Case

- make test system for newly updated parts of the program.
- once fixed parts are tested, run black box testing once again to ensure product with error free result.

7.3.3 Unit Testing

(rephrase) Unit testing is a level of software testing where individual units/ components of a software are tested. The purpose is to validate that each unit of the software performs as designed. A unit is the smallest testable part of any software. It usually has one or a few inputs and usually a single output.

Test Case

- Create a test system with three employee accounts. One who enter either wrong username or password and verify login error message. One who successfully login and claim the code in the list. One who will login and claim several codes from the list.
- Create a test system with three manager accounts. One who either wrong username or password and verify login error message. One who will login to check if there are enough code in the database. One who will manually make number of code to generate in the database.

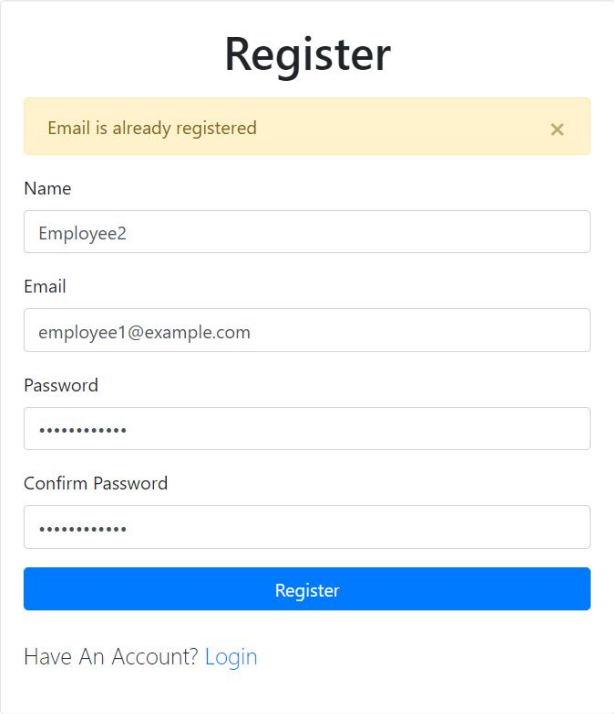
7.4 Test Results

Below a sample of passed test cases, along with their output on the web application is presented. Similar test cases have been omitted for brevity, but further testing output can be found on the accompanying slideshow presentation.

7.4.1 Verification Tests Passed

Test Case: Users cannot re-register

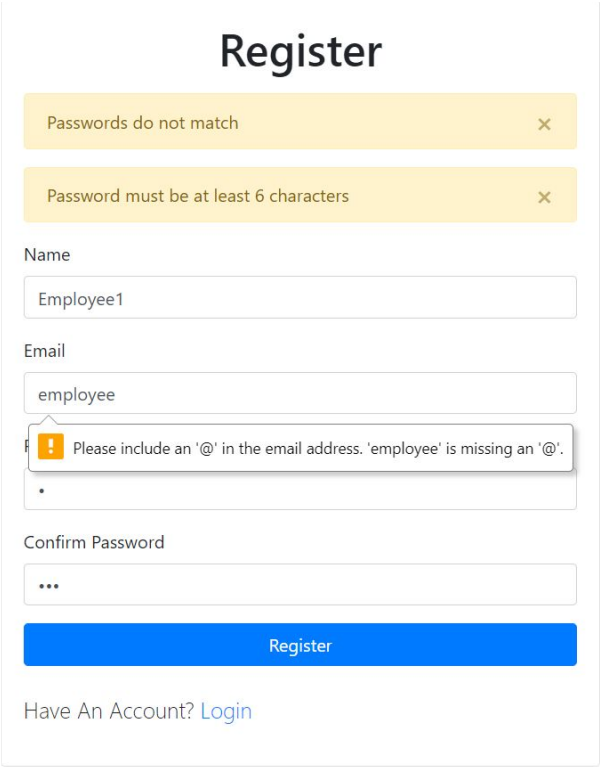
Passed (Output below)



Test Case: Passwords successfully hash and are not visible to admins

Test Case: All login inputs must adhere to requirements

Passed (Evidence Below)



Validation Tests Passed

Test Case:: Two users can have the same name but two accounts

Test Case: Code Dashboard is user-friendly. Easy sorting of used codes & easy search functionality

Test Passed (Evidence Below)

```
_id: ObjectId("5de4d80febc0cd2ebc92d1d0")
name: "Employee1"
email: "employee1@example.com"
password: "$2a$10$RooixdhSB.7AhKl3qGadhe5.YeWvoDZ7x2hKFWcscAWsauomRTpoy"
date: 2019-12-02T09:23:27.766+00:00
__v: 0
```

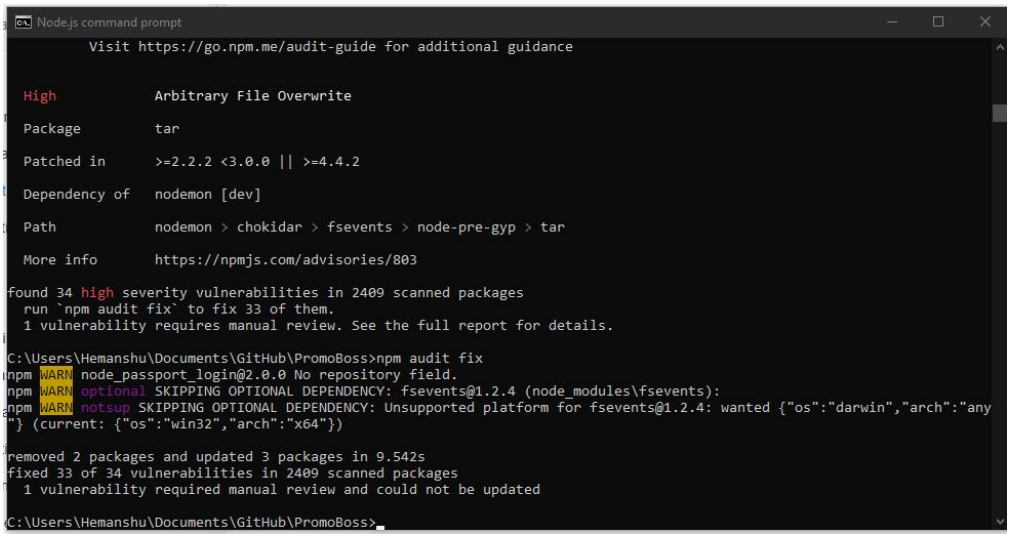
```
_id: ObjectId("5de4da3fad782637bc524c1e")
name: "Employee1"
email: "employee2@example.com"
password: "$2a$10$W4obKgvlZUW02vMX6ZDfo.3gywl7EzhxfE.SXaLSzlJkCYEClQ3LW"
date: 2019-12-02T09:32:47.525+00:00
__v: 0
```


7.4.2 Documented Errors

7.4.2a Error/Obstacle #1: System/Compiler Errors (Solved & Overcome)

Changed version to 4.4.8 on tar struct of package-lock.json. Then ran npm audit fix, change reverted to 4.4.1 again but error is gone. Issue is known and recurring, similar to a compiler or system issue. This issue is outside a developer's control. By noticing the issue and resolving the error (rather than ignoring the error, which some developers do), we are prepared to face such issues during hosting. Fortunately, our project is a web app, and as such we do not run the risk of the issue affecting client side operation (since we will host the website, the client either sees the entire page, or they do not).

https://stackoverflow.com/questions/55638180/how-to-fix-npm-package-tar-with-high-vulnerability-about-arbitrary-file-overwri



7.4.2b Error/Obstacle #2: Unforeseen Circumstances (Solved & Overcome)

Originally, we planned to host the website using one of our team member's account on "BlueHost." However, after attempting to launch a mock-up of our first release iteration, it was discovered that BlueHost does not have the required configurations to run NodeJS applications. As this discovery occurred rather late in the process, we opted for two solutions given our fast-approaching deadline. At the point of discovery, we were an estimated of 20+ hours behind. As discussed in section of the report, we revised our plan, and eliminated some of our desired requirements. Should we finish in a timely fashion– in accordance with our revised plan– then we will host the service using Digital Ocean. If we are unsuccessful in that approach, we will simply perform a local hosting through our local machine.

7.4.2c Error/Obstacle #3: Integration

Our persistent error was the failure to integrate our dashboard's Javascript to function alongside our server-side Javascript. This lead to the following errors:

- Error persists until expected deployment
- Led to delay & incompletion of planned tasks (i.e. metrics visualization, global hosting)
- JQuery module failed to export
- Client-side JS not visible with server-side JS running

7.4.2d Error/Obstacle #3: Button Mapping

Unfortunately, due to the lack of developer expertise, the following two small bugs were not corrected by the project completion date:

- Generate Codes button did not populate table while maintaining table integrity (data updates causes table to lose valid HTML format, which is needed for database front-end UI provided by the library we used)
- Upload file button was functional but stored as data structure

7.5 Metrics

7.5.1 Size Oriented

With this chart we can measure the metrics of our project and be able to compare to other projects metrics to determine where our efforts and results stand. An assumption we have made is that the cost of each hour of effort is $30.

Project	Lines of Code	Effort	Cost $(000)	Pages of Documentation	Errors	Defects	People
PromoBoss	1300	219	6570	60	3	2	4

Pages of documentation per KLOC (thousand lines of code):
Pages = 60 = 46.15 pages per KLOC
KLOC 1.3

Errors per KLOC:
Errors = 3 = 2.3 errors per KLOC
KLOC 1.3

Defects per KLOC:

Defects = 2 = 1.54 defects per KLOC
KLOC 1.3

8. Final Product

8.1 Abilities

- Privacy through secure credentials: Even our administrator/developer does not have access to any user's password. All passwords are hashed to ensure maximum privacy. Users can rest assured their credentials are safely stored and authenticated with PassportJS, the same authentication framework used to authenticate emails, Twitter, Facebook etc.
- Light-weight and portable: Entire database is stored on cloud (MongoDB Atlas)
- Robust search: all records can be searched to find specific employees to reward top-performers, etc.
- Sorting: All columns can be sorted, whether by used or available codes, or by name, date launched etc.
- Mobile responsive design
- Agnostic to organization

8.2 Constraints

- Limited to websites only. Website is cross-browser compatible, but not web-app etc exists

8.3 Missing Features

The functionalities removed were the additional data analysis of "promo codes" utilized and the visualization of said data analysis. The data analysis that was envisioned consisted of logging the following statistics of "promo code" use:

- Record of which employee utilized which codes
- Timestamp of codes used
- Timestamp of code generation (codes to be generated in batches, if a single code entailed a unique generation date, it would have its distinct timestamp and accompanying metrics)
- Expiration dates of codes
- A top performers table, outlining which employees had the highest number of codes claimed (from the inception of each code. All codes are generated in phases to ensure fairness and consistency of code generation date)

Chapter B: Umbrella/Support Activities

9. HR Section

9. 1 Hours Worked Per Team Member:

(Week 5)

This is a table representing hours each member spent in the making until the 5th week. Hemanshu, our team leader spent 35 hours in the process. Maria spent 32.5 hours in the process. Ruhao spent 34.5 hours in the process. Lastly, Abo spent 29.5 hours in the process.

Team Member	Hours
Hemanshu	35
Maria	32.5
Ruhao	34.5
Abo	29.5
Total	**131.5**

End of Project (Cumulative until Week 11)

Team Member	Hours
Hemanshu	65
Ruhao	58
Maria	45
Abo	42
Total	**210**

9.2 3-Week Plan

This is a detailed work record done by members in our team through week 4, 5, and 6. Each week recorded with the task assigned to each team member along with predicted time and actual time spent by each member on their assigned tasks.

Team Member	***Hemanshu***	***Maria***	***Ruhao***	***Abo***
Week 4 Tasks	Understand login system requirements and database requirements. Research technologies available, which front-end framework to pair with NodeJS (agreed upon back-end technology ahead of time). Enhance previous status reports.	Understand requirements. Research technologies available. Enhance previous status reports	Understand domain of cloud technologies, history of database jobs and automation trends. Enhance previous status reports.	Understand domain of cloud technologies, history of database jobs and automation trends. Enhance previous status reports.
Week 4 Accomplished	Login system. Enhanced previous reports. Presentation	Enhanced previous status reports. Worked on new status report. Presentation	Worked on new Status reports. Worked on presentation	Worked on presentation
Hours Allocated	10	10	10	10
Hours Worked	6	6	6	6

Week 5 Tasks	Provide material for presentation. Finish learning of tools related to JS frameworks involved. Compare MySQL vs MongoDB implementations Develop three week plan. 1 hour mid-plan meeting	Research continued	Research continued. Develop Prototypes using PhotoShop Develop three week plan. 1 hour mid-plan meeting	Research continued
Hours Allocated	9.5	7	9	7
Hours Worked	8.5	6	6	6
Week 6 Tasks & Accomplishments	Begin implementation of login system and attached database system.	Begin implementation of code generator and code database/data structure	Create presentation for impact of cloud based distributed databases	Create presentation for impact of cloud based distributed database
Hours Allocated	7	7	7	7
Hours Worked	7	7	7	7

9.3 Earned Value Analysis

Earned Value Analysis is a method of measuring the progress of the project at a given point in the timeline. This type of calculation provides a common value scale for every task, regardless of the type of work being performed. The following calculation is our Earned Value Analysis.

Week 05:

131.5 hours = 0.82 160 hours OR 131.5 hours = 1.64 80 hours	1 person per month working full-time OR 2 persons per month working part-time (20 hours a week, 80 hours a month)

Plan to be at 3 out of 5 milestones (Learning phase): 60% completed
Currently at 2 out of 5 milestones (Database phase) which indicated that 50% of the project is completed

Total hours = Current Hours Planned Completeness = 131.5 hrs 60% Total Hours = 219 hrs	
SPI = BCWP BCWS 0.50 = BCWP BCWP = 109.5	SV = BCWP - BCWS SV = 109.5 - 131.5 SV = - 22

The calculations indicate that our team is 22 hours behind schedule on November 1.

Week 11:

Planned to be at 100% completeness. An extra hour was added to the project as as revisions in program design and hardware failures caused a portion of completed work to be redone. Thus, even with improved productivity through better communication, 220 hours was total expected time.
Currently at 92%

<table>
<tr><td colspan="2">Total hours = Current Hours
Planned Completeness
= 220 hrs
100%
Total Hours = 220 hrs</td></tr>
<tr><td>SPI = BCWP
BCWS
0.93 = BCWP
220
BCWP = 210</td><td>SV = BCWP - BCWS

SV = 210 - 220

SV = - 10</td></tr>
<tr><td colspan="2">Equivalent to two person-months</td></tr>
</table>

The team is 10 hours behind schedule. In an effort to meet the deadline, a few planned functionalities have been removed. Integration with Tableau and later on charts.js was removed due to time constraints.

10. Risk Mitigation Monitoring Management (RIMMM)

<table>
<tr><th colspan="3">Risk Information Sheet</th></tr>
<tr><td>Date: 10/29/19</td><td>Probability: 90%</td><td>Impact: High</td></tr>
<tr><td colspan="3">Description:
Communication difficulty leading to missed group meetings.</td></tr>
<tr><td colspan="3">Refinement/context:
Subcondition 1: Only 50% of our group have efficient coding skills and experience

Subcondition 2: 75% of our group are working part time

Subcondition 3: Conflicting schedules</td></tr>
<tr><td colspan="3">Mitigation/monitoring:
1. Group leader will try to touch base with each member as often as possible
2. Try to go over everyone’s schedules weekly and plan meetups accordingly
3. Educate non-coders throughout the project phase about the coding tools used</td></tr>
<tr><td colspan="3">Management/contingency plan/trigger:
Cut extra features, such as sign up page or hosting the website online in order to get the other parts of the program polished. Also, use electronic means of communication for group meetings whenever possible instead of waiting to meet in person to go over everything.
Trigger: If mitigation steps was unproductive by 11/01/19.</td></tr>
<tr><td colspan="3">Current status:
11/15/19: Mitigation steps initiated.</td></tr>
<tr><td colspan="2">Originator: S. Said</td><td>Assigned: R. Lou</td></tr>
</table>

11. QA-Quality Assurance

To achieve quality outcomes, the team meets in person two times a week. On Friday, the team meets after class during the lab portion and goes over the final product for the week and assign tasks for the next week. The team meets again on Thursday to go over work done and complete any last-minute things need to be done, or help out teammates that may be having difficulty with a certain aspect. Throughout the week, to touch base, the team keeps in contact over a group chat on Whatsapp. Time spent is kept track of individually, and once a week on Friday before new tasks are allocated, the team informs the team leader of how many hours were spent and doing what. Team lead fills it in the master

hours spent chart. Review of each team member's work is done on Thursdays to gauge progress and whether tasks will be done on time for Friday's deadline.

In the beginning, team coordination and communication proved difficult as the method of communication used was verbal. An effective method for our team since has been to reiterate due dates and summarize tasks that need to be done in the group chat as well as filling it out in chart of tasks assigned. What has helped a lot is checking that chart frequently to make sure all subtasks are on track to expected completion. Due to the lengthy learning phase, and the lack of development expertise, we tried to best utilize the existing skill sets. Although only half of our team possessed development experience, with only the team leader who understood the framework during the learning phase, tasks were delegated in an attempt to maximize input from the group's diverse backgrounds. Our designer provided opinions on choice of web styling and our second documenter/writer helped brainstorm ideas for additional use-cases.

11.1 Lessons Learned

As a team, we found that we can accomplish higher quality work when we arrange face-to-face meetings– rather than relying on our chosen communication channels (Whatsapp group chat). Accomplishing tasks and effective decision-making using electronic methods like group chat often caused miscommunication and noticeable declines in productivity. As a result, our group has decided to adhere to Scrum's manifesto and conduct 24 hour daily scrum meetings. Additionally, due to the approaching deadline, we have added in twice-a-week face-to-face meetings and formalized Sprint Review sessions as well.

During Friday lab, we assign tasks and have a sprint review (we shortened the recommended Sprint session from two weeks to a single week, due to time constraints). During this review, we assign tasks to be completed for next Thursday. On Thursdays, depending on how much has been accomplished, we reevaluate and set more realistic expectations of delivery for successive sprints.

12. SW Engineering Process Model

12.1 Methodology: Agile

Due to the size, budget and time constraints of this project, it was clear that an Agile methodology was required. Furthermore, as a team, our programming ideologies we closely aligned with the Agile manifesto. Despite the amateur programming experience of the overall team, each of us agreed that our application is one where construction activities are the most vital. The Waterfall Model is bureaucratic by design, but even the UP and evolutionary models do not truly solve the issue of high software project failure (70-80% of all projects). After consulting with each other and realizing that following a broad Agile methodology was ineffective, we opted for the Scrum process framework.

12.2 Process Framework: Scrum

Upon choosing Scrum, we began to incorporate team evaluation. We began with assigning Fridays as both sprint review meetings and sprint goal meetings. Sprint goal meetings are conducted for tasks to be met by following Thursdays.

Although a true strict adherence to SCRUM was not possible, given that the team only had access to two developers, every attempt was made to follow any and all SCRUM processes that were logical to our situation. One might question why SCRUM was chosen by the team, knowing that it was not realistic to follow the software engineering principles strictly, and knowing that a novice team such as ours wld benefit from as much guidance as possible. Our justification for following SCRUM is as follows:

- From the very beginning of the project, it was clear that this project entailed a large number of unknown quantities. Our group was not familiar with each other to any degree, and thus the skillsets of each member were completely unknown. The only indication of skill and talent distribution was through
- Small size made agile approaches to software

12.3 Methodology Revision: Why SCRUM Proved to be Nonoptimal

Although SCRUM served its purpose in getting the team ready to "hit the ground running", it was slowly realized that it was not useful to solely rely on one software process framework. As outlined above, the individual skill sets was unknown to the team. As such, during development, it was discovered that the assigned co-developer did not understand the chosen framework for learning. As a result, it was decided that all design and analysis models would be redesigned, so that an MVP (minimum viable product) could be completed consisting of core functionalities.

12.4 Daily Scrum Questions

What sub-tasks did you accomplish?

What obstacles caused the delay?

How do you plan on accomplishing tomorrow’s task set?

In keeping with SCRUM framework fashion, we adhered to daily progress updates for each and every designated sprint.

13. Integration

We believe that the core of any integration strategy relies on its communication efficiency. From the very beginning, a strong commitment to mitigate miscommunication and have documented task lists was made.

We utilized a bottom-up integration strategy for integration of all parts. This was done to ensure construction activities do not delay the integration of user requirements, design and analysis. With any

progress made during the coding or challenges we faced, we had to ensure that when a solution was found, alternative strategies were developed on how to proceed with the design. However, these did not deter us from the overall goal of what the application was trying to achieve.

For example, we considered different methods in which new employees are able to get registered into our database. We thought of the IT department being able to hard code new users into the system in the back end, or give the managers the ability to register new users, or lastly allowing users to register on the website themselves. We had to ensure that whatever method we go forward with made sense for any type of business solution. After figuring out our technical limitations, our main developer decided to go forward with allowing users to register themselves. This is where we all had to double check the option we were going forward with and decided that since our application is not simply a public website that anyone will have access to or register to, we should set a certain limitation to keep it exclusive. We implemented the Employee ID field which must be entered and is uniquely assigned to the users by their own employers outside of our application, which is very common among businesses. This was a significant example of integration during our development process, where we had to ensure that our coded product and our business case vision were going hand in hand.

14. Reflective Assessment

14.1 Team Reflection (Week 5)

After assessing our sprint review meetings and compiling a comparison between “hours assigned” against “hours worked”, we concluded that our team is behind schedule. In order to offset the unforeseen delays caused by sickness, employment and academic struggles, we have decided to aggressively add hours for construction activities. To better manage this process as novice managers, we subdivided our sprints using the famous “divide and conquer” strategy. Furthermore, as discussed above, we opted to meet more frequently in person, as this was undeniably the most successful communication strategy in our case. As a team, we will continue to evolve app requirements to ensure all members are “on the same page” regarding functionality.

14.2 Individual Reflection

To reach both an effective individual and team reflection, as project manager I was inspired from the process metrics discussed in the classic text by Pressman and decided to create a less technical– but equally detailed– model of individual performance. When I say performance, I refer not to metrics assigned to individuals or metrics used to evaluate individual performance. Rather, I evaluate the “people” factors that influenced the constraints and decisions which affected the software development process. To do so, we used the popular MARS model of team performance. This model is often used by organizations across multiple industries, and is not limited to any domain.

Hemanshu Bhargav

Motivation: Creating a real-world data management application and productivity utility which is agnostic to organizations and utilizes industry-standard

Abilities: Functioned as the main developer due to prior experience in a plethora of programming languages, including the chosen framework Nodejs. Used knowledge of databases to originate all designs, including UML class diagram, architectures, analysis models and more. Performed all research to delegate development and design/analysis tasks. Solely created login/ registration system, database connection, sorting/searching functionalities of dashboard, code generation utilities and file upload functionality.

Role Perceptions: Team leader. Managed and create communication channels (slack was abandoned due to team consensus, Whatsapp proved to be more convenient and efficient). Realize team was behind schedule at multiple instances. Perform routine daily "checkups" to ensure progress of tasks by all group members. Aid in learning activities for all members. Create design and analysis models. Create and assign tasks according to team member's skill levels and schedules. Perform all research to strategize development. Constantly monitored progress of all team members.

Situational Factors: Responsible for originating all design models so additional models could be developed. When co-developer was unable to complete development tasks, forced to work hours above week's allotment to ensure product is ready to deploy. Was hired as a programmer during semester, and had transition period to leave current part-time job, increasing workload during week five (where actual output was lower for myself. Onwards actual output was on-par, if not above allocated tasks)

Abo Said

Motivation: Finding a working solution to real life workplace problem that has cost many sales.

Abilities: Presentation, light coding, problem solving, extracting main idea and supporting communication between group.

Role perception: Originator of the project idea, thus ensuring that the main vision was kept consistent throughout the development process. Worked on documentation, modelling, presentations as well as assisting remaining group members if needed. Attempt to learn and understand the coding procedures the main developers were using. Has first hand experience of the difficulty of finding promo codes when needed and so uses that predicament to drive the team to build the solution to the problem.

Situational Factors: 30 hour work weeks along with hour long commutes and 3 classes in the semester made it difficult to meet up and work on the project as often as desired.

Rose Ruhao Lou

Motivation: Managing UI design framework and usability of the project.

Abilities: Graphic Design, light coding skills, presentation, problem solving, assisting in documentation and research.

Role perceptions: UI management, Diagram developing, presentation creator, and documentation/status report. I was supposed to take after main coding was completed , adding CSS/design as additional feature.. However, as we were behind schedule, we ended up using bootstrap to polish the project

Situational Factors: No background in programming until few months ago. Entire class was a learning process. Consider course as very challenging despite giving personal best.

Maria Ashraf

Motivation: Creating an application using different technologies that can communicate. Having analytical capabilities to analyze the usage data collected.

Abilities: Problem solving, coding, productivity software.

Role Perceptions: Secondary Developer. Status reports writer. Created design models. Requirements were continuously updated, led to some confusion and redoing of models.

Situational Factors: As we were behind schedule (time constraints) and other challenges faced, online resources were used to learn and implement functionality. Reusability of code principles. Some features of application were left out.

14.3 Team Reflection (Week 11)

As a team, we felt that the following factors created ambiguity and delayed development:

- subgoals were hazy
- subsystems were unclear and continuously changing, causing multiple code revisions, ran out of time as deadline approached
- communication: some members respond, some don't. Fortunately, our team was of the cooperative nature, however, communication barriers proved to be an obstacle nonetheless .... As soon as the problem was realized, our team decided to increase the frequency of meetings, to ensure that all members were understanding what is required of them. Confusion of duties was not the only hurdle to be overcome, as a lack of knowledge was later discovered to be the true culprit.
- Set clear communication strategies. In future: document all communication. We planned to do so but kept everything informal to not pressure members and make everyone feel welcome to ask questions. Resulted in lax deadlines, miscommunication. A slightly more formal approach would have been beneficial when external factors caused members to be unresponsive. I consider this one of my failing as project manager, because although we were able to deal with the issue without involving the professor, we did lose a significant time
- What do we want to learn? Temporary flip-flop between visualization through Tableau and Chart.js (the constraints that lead to this are discussed above)

14.4 Going Forward

In hindsight, we believe the developing more formal, strict deadlines with accompanying consequences would have prevented the button mapping failures discussed above. Our advice to future students is to:

- Anticipate unforeseen circumstances, regardless of likelihood (both developers had a laptop crash, hiring during semester etc.)
- Strict deadlines for subtasks
- Evaluate skill sets for all team members
- Realistically planning tasks based on the skill set evaluation

Challenges overcome:

- Used a variety of file sharing utilities and communication channels:
    - File Sharing through Github & Google Drive
    - Communication through Slack & Whatsapp
- Perform testing, design models together so that all members are “on the same page” so to speak
- Due to the lack of experience in our team and non-technical backgrounds of some of our team members, we strove to develop an open communication strategy where all updates were pushed to google drive and/or github (which was later rejected to ensure all team members could easily make changes and adapt documentation along with construction activities simultaneously)

14.5 Challenges

- 1) Lacked any experience with Jquery and little experience with JS. All prior JS experience was using vanilla JS, did not know of Jquery tools. No JS development since approximately 20 months
- 2) Prior node.js experience was 2 years ago, and at that time was done in a similar fashion to this project, and learned node.js with little JS experience, learning it only to discover its potential with routing capabilities (out of pure interest).
- Cracked laptop → had to relearn from scratch, no notes regarding development platform, this is why node alone was chosen over a combination of React and MySQL. A more robust front-end was scrapped entirely.
- Both developers lacked adequate JS experience
- One member switched part-time jobs which required its own learning phase. This learning phase resulted in reduced commitment to original plan.
- Two out of the four members are not computer science students
- One member was asked to increase hours at work and was taking a 60% course-workload. As a project manager, the decision was made to not punish the member for external, unforeseen factors which were out of their control. Rather an effort was made to cooperate with the team member and their presence in meetings was made optional– instead said team member was to check group updates whenever possible. Lack of time resulted in resulted in reduced commitment to original plan.
- Two members faced greater, unexpected difficulties in their other courses. Academic struggles in those course resulted in reduced commitment to original plan.

- Lack of technical experience across members, as a result removed the following:
    - Removed tableau due to lack of integration of mongodb cloud and tableau
    - Researched other visualization libraries, selected chart.js but did not have time to develop significant metrics due to simplistic design of database
    - Analysis revision: realized more code was not needed, did not need code managers etc.
- Were assigned responsibility of documentation. However, above stated circumstances resulted in a new division of labour. Developers increased work-hours on documentation/design

Revision of Design Models- Post Final Construction Activities Stage

- while developing, realized robust database with multiple tables and managers was needed
- instead revised model to codes not being structural entities and modified to boolean values, USED for ‘true’ and AVAILABLE for ‘false’
- As we manually created sorting of each column we discovered jquery’s datatables, and found the built-in options
- All design models were revised to incorporate this feature
- Design addition: to compensate for lack of visual and robust data manipulation, a feature to upload or input codes was added as a user requirement

15. Appendix

15.1 Meeting Minutes

This chart shows what was discussed in every in-person meeting held. As you can see, in the later half of the semester, we realized how far behind we were and began holding two meetings a week instead of one.

Week Of	
13/09	**Announcement:** Team formation **Discussion** Formed group and exchanged contact info (unofficial meeting). Discussed skill level of each member. **Goals Achieved:** Teams formed **Goals for Next Meeting:** Think of product ideas to discuss with team
20/09	**Announcement**: Proposal Formed **Discussion** Formed business proposal early-on to gauge feasibility and reactions of each team member. Created communication channels & collaboration tools for status reports. **Goals Achieved:** Created a business proposal with well-formed idea **Goals for Next Meeting:** Figure out tools and technologies to use for application
27/09	**Announcement**: Begin Learning Technologies to be Implemented **Discussion** Revised project ideas based on research of existing programming technologies. Divided responsibilities based on

	labour cost and designated roles of team members. Discussed details based on research **Goals Achieved:** Assigned responsibilities to each member. Finalized a preliminary list of technologies to use **Goals for Next Meeting:** Find a development methodology that fits our type of web application
4/10 (Oct.4 Rotated Project Manager)	**Announcement:** Began rotations of project manager **Discussion** Discussed individual strengths and experience as we get closer to developing the project. Pitched idea for Scrum-like development process. Worked on status report together to have a more open discussion regarding functionality and documented it. **Goals Achieved:** Settled on Scrum-like development process Finished status report together **Goals for Next Meeting:** Plan out tasks and their priority level
11/10 (Oct.11 Rotated Project Manager)	**Announcement**: Division of Labour **Discussion** Discussed division of labour. Discussion of introduction of git and expansion of collaborative tools. Finalized development priority as login-system and database integration by first week of November. Presentation allocated to members. **Goals Achieved:** Divided labour. Prioritized login system and database over other tasks **Goals for Next Meeting:** Set schedule for learning technologies

18/10	**Announcement:** Scheduling learning **Discussion** Finalized technologies that will be used. Set out a schedule of who will be learning what. Updated previous status reports for consistency as software project continued to change. **Goals Achieved:** Updated list of technologies. Updated previous status reports **Goals for Next Meeting:** Figure out how technologies communicate and connect
25/10	**Announcement:** Learning continues **Discussion** Continued learning technologies and figuring out how they connect with each other. Discussed as a team what we learned and shared our thoughts regarding its implementation. **Goals Achieved:** Learned more about technologies and their limitations **Goals for Next Meeting:** Finalize which technologies to use
1/11	**Announcement:** Technologies Used List Updated **Discussion** Created diagrams for process flow of our application. Updated list of technologies used as limitations cropped up. Some unable to communicate with each other or required too much time to implement. They were discarded in favour of ones easier to implement. MongoDB was used instead of MySQL. **Goals Achieved:** Finalized software and other technologies. Created process flow diagrams

	Goals for Next Meeting: Find if we are on track in development process (GANTT Chart)
7/11	**Announcement:** First Meeting of Twice Weekly In-Person Meetings **Discussion** Updated SwimLane diagram with new steps after discussion. Added Use Case Diagrams for sub-functions. Planned out more twice weekly meeting moving forward. Inception of login page and its functionalities formed. Email address would be used as a unique user value instead of name. **Goals Achieved:** Updated previous diagrams and added additional diagrams. Settled on unique value that will be used for authentication **Goals for Next Meeting:** Meet twice a week. Fast track development
8/11	**Announcement:** Re-evaluating at the half-way point **Discussion** How to best present our project idea to the class. Worked on creating slides and walking through steps of application's functionality. Created some sample pages for visual guide. Discussed difficulties faced when trying to connect node.js login page to MongoDB at back-end. **Goals Achieved:** Created presentation for class **Goals for Next Meeting:** Make sure product is on track with business case and requirements set
14/11	**Announcement:** Reframed functionalities **Discussion**

	Discussed functionality again as requirements changed, created several different design models to better understand the process. Went over business case to make sure our product was on track with our business case requirements. Worked through the login page difficulties the team faced. **Goals Achieved:** Created design models **Goals for Next Meeting:** Update previous reports to better reflect new design models
15/11	**Announcement:** Keeping previous status reports up to date with changing requirements **Discussion** Discussed final document and how to update previous status reports to reflect changing requirements. Added more details to our reporting to provide in depth look at our process as well as what we were trying to achieve. Worked on login page and the several web pages the final product will have. **Goals Achieved:** Updated previous reports. Made progress on web pages for application **Goals for Next Meeting:** Start testing process
21/11	**Announcement:** Testing begins **Discussion** Learned proper way of carrying out testing and documenting. Planned out testing procedure and created models such as Pipe and Filter, and Model View Controller. Discussed more features to be left out because of time constraints like hosting on a live server. **Goals Achieved:** Learned how to test and document results

	Goals for Next Meeting: Start testing and writing down results
22/11	**Announcement:** Testing continues **Discussion** Worked on Testing Procedures and errors that occured. Documented and enhanced design as application was adjusted to mitigate errors discovered. **Goals Achieved:** Updated designs to better reflect application's functionalities **Goals for Next Meeting:**
28/11	**Announcement:** Planning final report **Discussion** Planned out final report and division of labour based on strengths. Kept channels of communication open to make sure analysis were consistent for each section even if done by different members of the team. Enhanced tables to be displayed in each webpage and their functionalities. **Goals Achieved:** Enhanced application web pages. Finished planning final report **Goals for Next Meeting:** Document challenges and add to final report
29/11	**Announcement:** Constraints faced **Discussion** Started putting final report together. Reflected as a group on circumstances that led to the team being behind schedule and the challenges faced. Conducted testing and documented errors faced. **Goals Achieved:** Documented tests and errors

	Goals for Next Meeting: Finish application
5/12	**Announcement:** Finalizing Application and Report **Discussion** Created list of tasks left to complete for final presentation and report. Divided up the list based individual strengths. Conducted additional testing. Made UI look consistent across all pages. Tried to connect static pages and database one last time. **Goals Achieved:** Had a final product with some functionalities **Goals for Next Meeting:** Finish presentation and final report. Make a brochure
6/12	**Announcement:** Final Presentation and Report **Discussion** Finalized report to be handed in and organized into proper section headings. Discussed look of brochure and what we want it to say. Practiced presentation and created visuals to present. Went over the functionalities left out due to constraints and risks. Discussed what can be done moving forward with the application and what we learned from this process. **Goals Achieved:** Presented product and final report

15.2 Hours Worked

This chart shows the amount of hours spent by each team member per week. This is allocated based on time spent working on documentation, coding as well as learning the technologies.

Legend M = Meeting R = Research/Coding S = Status Report		**Labour Cost (hours)**			
Week #	**Week Of**	**Hemanshu**	**Rose**	**Maria**	**Abo**
Groups Formed	13/09-20/09	1 M 0.5 M/R	1 M 0.5 M/R	1 M 0.5 M/R	1 M 0.5 M/R
1	20/09-27/09	2 S/R 1M	1 MS	1M	1M
2	27/09-04/10	4 MSR	4 MS	1	1 MS
3	04/10-11/10	4 MSR	4 MS	2 R	-----
4	11/10-18/10	2 MSR	2 MS	2 MS	2 MS
5	18/10-25/10	2 MSR	2 MS	1 MSR	1 MS
6	25/10-01/11	6 MSR	6 MS	6 MS	6 MS
7	01/11-08/11	8.5 MSR	8 MS	6 MSR	6 MS
8	08/11-15/11	6 MSR	6 MS	4 MS	4 MS
9	15/11-22/11	6 MSR	6 MS	4 MS	4 MS
10	22/11-29/11	12 MSR	9.5 MSR	4 MSR	7 MS
11	29/11-06/12	10 MSR	10 MS	6 MS	8 MS

15.3 Tech Specs

NodeJS: The Backend powerhouse

What is node.js? Why did we choose it?

There is no shortage of programming languages for web development, nor is there a shortage of server-side web scripting languages. So, the creation of node.js may seem like a fad. However, this framework is truly game-changing. To understand the reason behind this, we must look at the underlying foundation in which node.js is written.

Node.js is written in C++ and “piggybacks” off of chrome v8 engine. This allows node.js to do something that was perhaps never done successfully in modern times before.

This is why it can write JavaScript on the server side, because it uses the same Google engine that is used in JavaScript when it is written in client side (most client-side web development is written in JavaScript) googles engine –taught JS is client side written in HTML, so the question becomes, how is this so supposedly game-changing is it different, why can we write it on server side when its meant for client because it using google's v8 engine and allows the programmer to write in conjunction of the Node.js itself lets u write JavaScript server side-that’s why u have so many functions that one can’t write client side, symbiotic relationship

- Also using template and dynamically creating content depending where user navigates the page included bootstrap in a different way, not in head but in JavaScript server-side setup itself
- -CSS is included dynamically great for several different page requirements, can include different CSS style sheets per page without overcomplicating classes and ids all one external style sheet
- -Applications: use of relational database of MongoDB for login interfaces with custom dashboards based on information received from Cookies
- -Can write JS server side and client side is very powerful to developers – fluency between development
- -Instead of PHP or PERL server and client-side JavaScript and CSS, now all JavaScript on both server side and client side

Middleware

Middleware is software which contains services for the party in question to function successfully. It has access to the request, response and next function parameters of the many instances of callback functions. You may notice, many of the dependencies that we create in the package.JSON file require us to write middleware; while none of the core modules do. Each middleware calls the next middleware which allows these multiple modules to work in unison to make on node.js application or provide some multiple-featured service which feels like is all just one package.

More generally speaking, middleware is a pipeline or channel that acts as the medium on which clients and servers can be linked. Middleware technologies include a wide array of software and services which connects front-end programming (user-interfaces) with back-end programming (server-side, hidden from the user) in the distributed computing. Middleware is often used to provide the following services for connecting software:

1. Business Process Architectures
2. Message Brokers
3. Application Servers

4. Transport

We implemented the following middleware technologies:

- bodyparser
- express
- passport

We also attempted to create our own middleware to better integrate the dashboards and login system, however this was not possible due to time constraints.

One may argue that the use of the http module leads to something that could be accomplished with any server-side language, and node.js brings nothing new to the table. So, to truly use the power of node.js to ensure only a manager can see pages designed for managers, we discovered another framework within NodeJS: the ExpressJS framework, a framework which connects a web application's backend and frontend through dynamically routing pages (or rendering views).

ExpressJS: Helping backend frameworks connect

Routing

The power of Express lies in routing. to start we declare two variables to store ou routes. These will work with handlebars. We used handlebars in this tutorial as it is a common tool used in conjunction of node.js. Note: handlebars isn't the only option "Pug" is also used as well and more popular. However, the syntax is different from plain HTML and we wanted to keep this tutorial simple and focused on node.js

Rendering Views

Notice the lines of code highlighted in the image. These lines of code are where ExpressJS renders the views from the handlebar files (index.handlebars, page1.handlebars, and page2.handlebars) into the default layout.handlebars (see image below). From this we can see a simple example of the power of routing in Node.js. The index.handlebars serves as the homepage, it is located in a separate folder or directory. (This can be seen in the two variables "users" and index" which are initialized to the file pathways)

Path module

How does node.js know where my CSS is and where my folders are. Don't worry, node.js has that all figured out with the path module. It establishes the path between one's file system and plays a crucial role in routing (which holds the power of node.js' dynamic webpage creation).

Database Management: Mongoose & MongoDB

Although we had a hard time deciding whether to use SQL or MongoDB, further research quickly ended our indecisiveness. Due to the nature of our data, being of limited size and occuring in a key-value pair nature (at least in the manner we envisioned it), MongoDB theoretically held greater performance dividends. Needless to say, we never saw any latency from our MongoDB database connection.

However, understanding the nature of MongoDB and its connection with NodeJS was no trivial task. With a plethora of distributions available, we settled on MongoDB Atlas, a cloud distribution of Mongo. This decision was made in order to meet the core business requirements laid out at the beginning of the project. The web application was to be made as light-weight and portable as possible. A cloud distribution was the clear winner in both of these categories. Next, we had to devise a method to connect the database to our application. This fortunately was a simple decision, as Mongoose is a framework which is designed for this very task. Mongoose allows nodejs to connect to the MongoDB database, instantiate users on-the-fly and export their data via expressJS routing.

List of all Tools/Programming Languages

- NodeJS (backend and overall framework which houses all tools, chosen for routing abilities: even bare-bones NodeJS and ExpressJS implementations)
- ExpressJS (connects middleware, front-end and back-end)
- PassportJS (authenticates registration and login system against database, ensures only employees can access their pages, industry-standard authentication for email, Twitter, Facebook etc.)
- Bcrypt (hashes passwords so admins cannot see users' passwords)
- EJS (templating engine to host HTML through server-side JS pages)
- MongoDB Atlas (cloud database)
- JQuery (client-side validation)
- JQuery Datatables (table with adaptable sorting and searching features, and able to accept JSON data which would be outputted from MongoDB Atlas)
- Bootstrap/Bootswatch (consistent theme styling)
- HTML/CSS (basic web-development)

Editors & Platforms

- Visual Studio Code was used due to convenience of parameter descriptions and its split-screen layout with terminal/command prompt control. Originally Atom was used due to Github integration, but as development became concentrated with one individual, VS code proved to lead in performance
- Tested on Windows, MacOS, and multiple browsers using VS Code
- Google Chrome was used as the primary development tool due to its plethora of developer tools
- Github & Google Drive for collaboration (shared folder so all members have access to all information)
- Slack/Skype & Whatsapp for communication (convenient, on-the-go communication through Whatsapp and screen-sharing during video calls with Skype/Whatsapp)

16. Bibliography

Black Box Testing. (2018, March 3). Retrieved from http://softwaretestingfundamentals.com/black-box-testing/.

Unit Testing. (2018, March 3). Retrieved from http://softwaretestingfundamentals.com/unit-testing/.

System Testing. (2018, March 3). Retrieved from http://softwaretestingfundamentals.com/system-testing/

NodeJS. (2018, April 18). Retrieved from https://www.freecodecamp.org/news/what-exactly-is-node-js-ae36e97449f5/

The following GitHub repository was used (repository accompanied YouTube tutorial) https://github.com/bradtraversy/node_passport_login